Datenmodellierung und Datenbanken

Relationale Modellierung

Warum Datenmodellierung entscheidend ist

Ein gutes Datenmodell sorgt für:

- konsistente Daten,
- klare Geschäftslogik,
- wartbare Systeme,
- bessere Query-Performance.

Häufige Folgen eines schlechten Datenmodells

Ein schlechtes Datenmodell führt oft zu:

- Redundanz,
- widersprüchlichen Daten,
- teuren Workarounds in Abfragen.

Stadien der Datenbankprojektierung

- **Konzeptioneller Entwurf** (oder: fachliches Datenmodell)
- **Logischer Entwurf**
- **Physischer Entwurf**

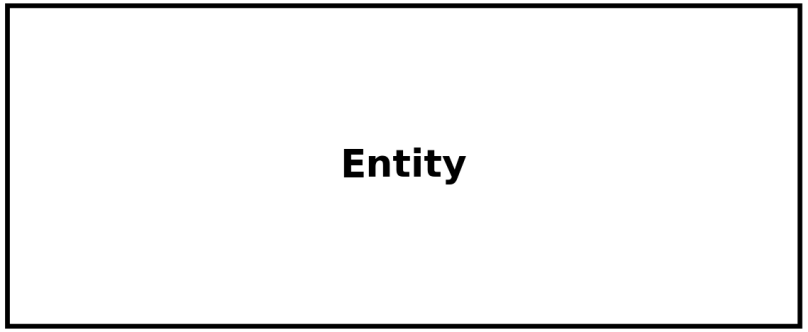
Konzeptioneller Entwurf

- Aufbau eines semantischen Modells der Fachdomäne, also eines Informationsmodells auf höchster Abstraktionsebene.
- Ohne Orientierung an einer konkreten Datenbanksoftware oder einem bestimmten Datenmodell.
- Beschreibung der Informationsobjekte bzw. Fachbegriffe und der Beziehungen zwischen ihnen.
- Beschreibung von Integritätsanforderungen: zulässige Datenwerte und zulässige Beziehungen zwischen Daten.

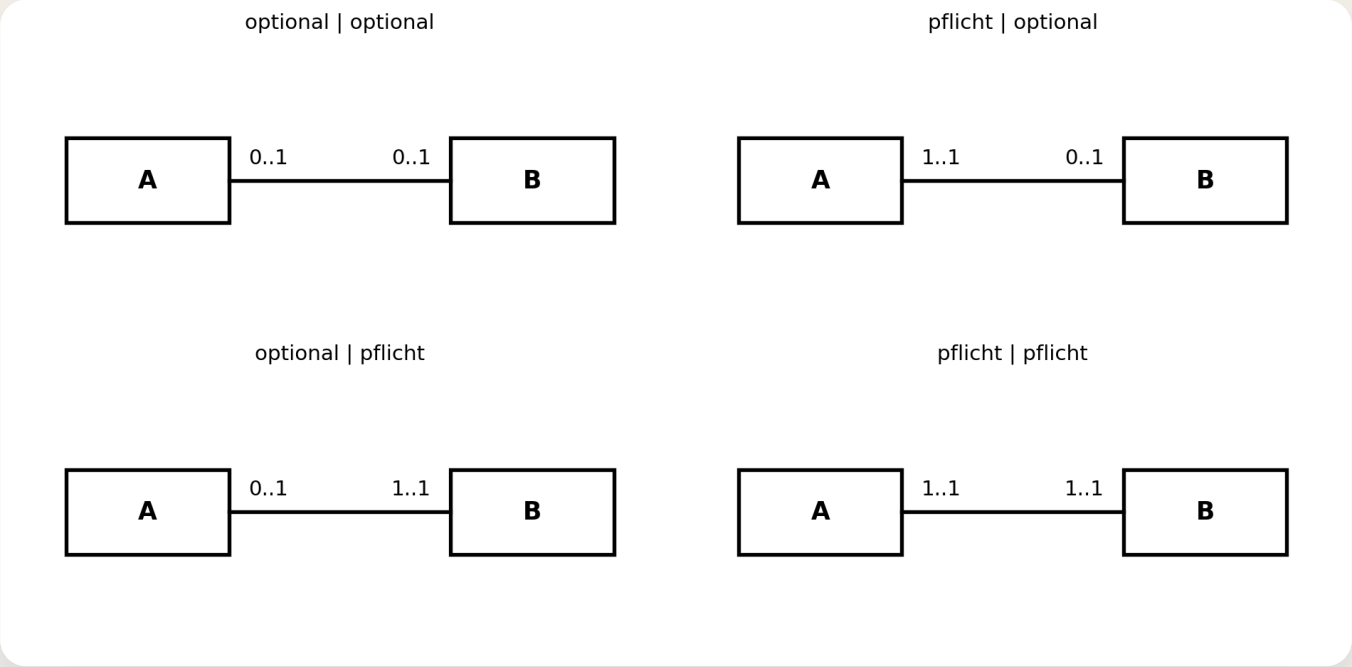
Entity-Relationship-Modell (ER-Modell)

- Das **Entität-Beziehung-Modell** (engl. *Entity-Relationship Model*) ist ein Modell der Fachdomäne.
- Es dient dazu, die **Semantik der Fachdomäne** auf der **höchsten Abstraktionsebene** darzustellen.
- Im Fokus stehen Entitäten und die Beziehungen zwischen Entitäten.

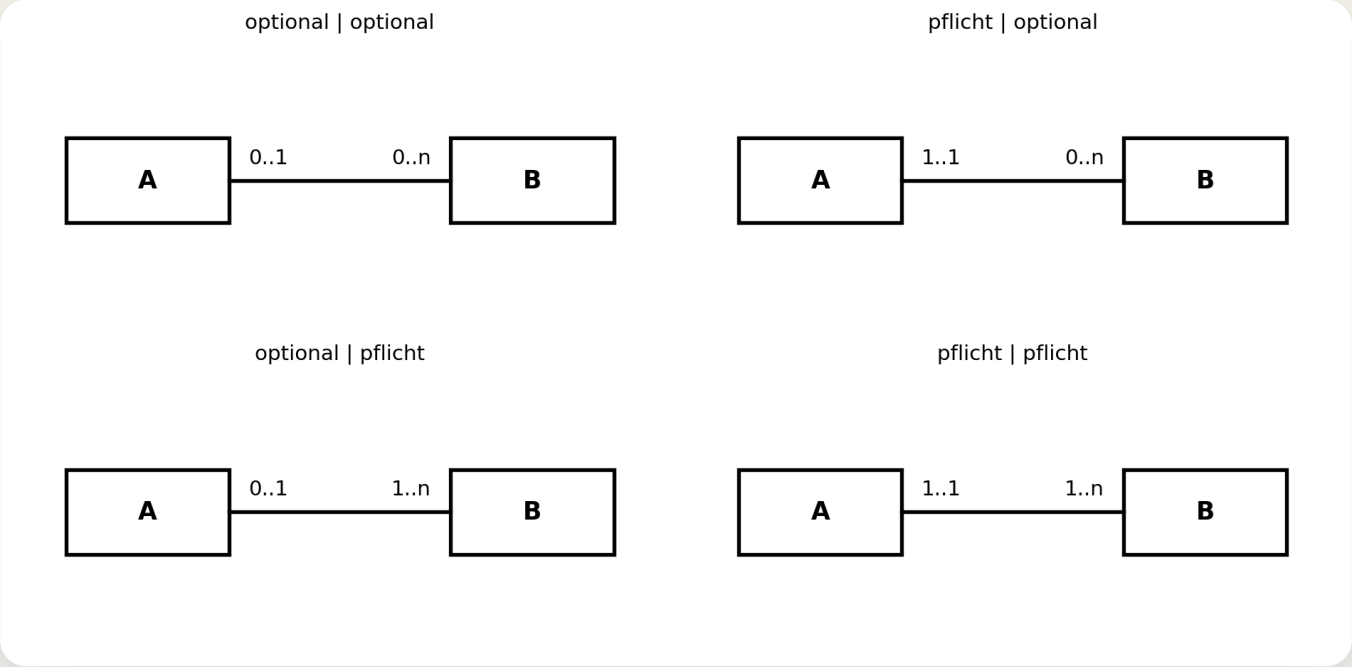
Entität



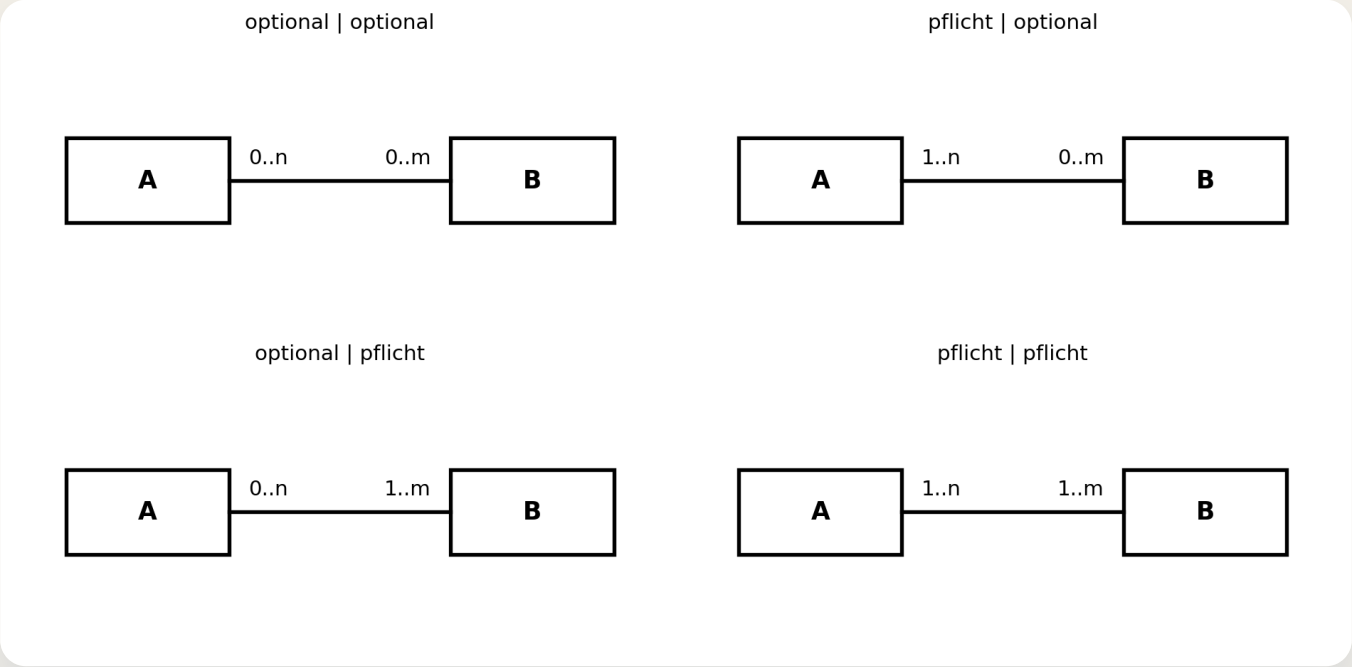
Beziehungen 1:1 (optional/pflicht)



Beziehungen 1:n (optional/pflicht)



Beziehungen n:m (optional/pflicht)



Beispiel: Fakultät, Gruppe, Student

Interpretation: Fakultät, Gruppe, Student

- **Fakultät - Gruppe**: Eine Fakultät hat `0..n` Gruppen, eine Gruppe gehört zu `1..1` Fakultät.
- **Gruppe - Student**: Eine Gruppe hat `0..n` Studierende, ein Student gehört zu `1..1` Gruppe.

Beispiel: Abteilung, Mitarbeiter, Projekt

Interpretation: Abteilung, Mitarbeiter, Projekt

- **Abteilung - Mitarbeiter**: Eine Abteilung hat `0..n` Mitarbeiter, ein Mitarbeiter gehört zu `1..1` Abteilung.
- **Mitarbeiter - Projekt**: Ein Mitarbeiter arbeitet an `0..n` Projekten, ein Projekt hat `0..n` Mitarbeiter.

Logischer Entwurf

- Erstellung eines Datenbankschemas auf Basis eines konkreten Datenmodells, z. B. des relationalen Datenmodells.
- Für das relationale Modell ist das logische Modell eine Menge von Relationsschemata, meist mit Primärschlüsseln sowie Beziehungen über Fremdschlüssel.
- In der Phase des logischen Entwurfs wird die Spezifik des gewählten Datenmodells berücksichtigt, die Spezifik eines konkreten DBMS kann jedoch noch unberücksichtigt bleiben.

Beispiel (Fakultät, Gruppe, Student)

```
FAKULTAET(
  fakultaet_id PK,
  name
)

GRUPPE(
  gruppe_id PK,
  bezeichnung,
  fakultaet_id FK -> FAKULTAET.fakultaet_id
)

STUDENT(
  student_id PK,
  vorname,
  nachname,
  gruppe_id FK -> GRUPPE.gruppe_id
)
```

Beispiel (Abteilung, Mitarbeiter, Projekt)

```
ABTEILUNG(
  abteilung_id PK,
  name
)

MITARBEITER(
  mitarbeiter_id PK,
  vorname,
  nachname,
  abteilung_id FK -> ABTEILUNG.abteilung_id
)

PROJEKT(
  projekt_id PK,
  titel
)

MITARBEITER_PROJEKT(
  mitarbeiter_id FK -> MITARBEITER.mitarbeiter_id,
  projekt_id FK -> PROJEKT.projekt_id,
  rolle,
  PK (mitarbeiter_id, projekt_id)
)
```

Physischer Entwurf

- Umsetzung des Datenbankschemas für eine **konkrete** DBMS-Plattform.
- Berücksichtigt die Spezifika des gewählten Systems, zum Beispiel:
- Namenskonventionen und Namensgrenzen für Datenbankobjekte.
- Unterstützte Datentypen und deren systemabhängige Eigenschaften.

- Auswahl von Speicher- und Zugriffsverfahren.
- Aufteilung der Datenbank auf Dateien, Tablespaces und Geräte.
- Verteilung und Optimierung von Datenzugriffspfaden.
- Erstellung und Feinabstimmung von Indizes.
- Kurz: alles, was an das konkrete DBMS gebunden ist.

```sql
CREATE TABLE faculty (
 id int(11) unsigned NOT NULL AUTO_INCREMENT,
 name varchar(255) DEFAULT NULL,
 PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

CREATE TABLE `group` (
 id int(11) unsigned NOT NULL AUTO_INCREMENT,
 faculty_id int(10) unsigned DEFAULT NULL,
 name varchar(255) DEFAULT NULL,
 PRIMARY KEY (id),
 KEY faculty_id (faculty_id),
 CONSTRAINT faculty_id FOREIGN KEY (faculty_id) REFERENCES faculty (id) ON DELETE SET NULL ON UPDATE
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

CREATE TABLE student (
 id int(11) unsigned NOT NULL AUTO_INCREMENT,
 first_name varchar(255) DEFAULT NULL,
 second_name varchar(255) DEFAULT NULL,
 last_name varchar(255) DEFAULT NULL,
 group_id int(10) unsigned DEFAULT NULL,
 PRIMARY KEY (id),
 KEY group_id (group_id),
 CONSTRAINT group_id FOREIGN KEY (group_id) REFERENCES group (id) ON DELETE SET NULL ON UPDATE SET NU
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```